AYDIN ADNAN MENDERES UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CSE 419 ARTIFICIAL INTELLIGENCE
Assignment#2

Prepared by: Cavit Eren YILDIZ

No: 201805001

1. Introduction

This project focuses on developing a smart home agent using the **Logic-Thought** approach, which aims to formalize human-like reasoning through structured logic. The goal is to enable machines to "think" and act according to rational standards.

The smart home agent observes user commands (entered as text), interprets them into **propositional logic facts**, and executes corresponding **actions** based on defined **rules**. For example, if a user types "I'm cold", the system interprets this as cold_weather and turns on the heater accordingly.

2. System Architecture

a. User Input

- The user types natural language commands into the system.
- Speech recognition is ignored in this version.

b. LLM Interpreter (Gemini API)

- Uses a large language model (Google Gemini) to convert user input into a predefined **logical fact**.
- Model: gemini-pro (or similar)

c. Facts & Rules

- The agent maintains a set of known **facts**.
- Each fact can trigger a **rule**, which maps to an **action** (e.g., cold_weather -> turn_on_heater).

d. Action Executor

- If a fact matches a rule, the corresponding action is printed to the console.
- Physical smart devices are not controlled in this version, but the architecture supports future integration.

3. Defined Facts and Actions

Facts and Corresponding Actions:

{ "cold_weather": "turn_on_heater", "hot_weather": "turn_on_ac", "no_one_home": "turn_off_all_devices", "dark_room": "turn_on_lights", "bright_room": "turn_off_lights", "sleep_time": "close_curtains", "wake_time": "open_curtains", "wake_up_time": "open_curtains", "hot_enough": "turn_off_heater", "cold_enough": "turn_off_ac", "leaving_home": "lock_door", "arriving_home": "unlock_door" }

4. **Code Structure**

File	Purpose
main.py	The entry point of the application. Takes user input, triggers actions.
interpreter.py	Sends user input to Gemini API and retrieves the interpreted fact.
rules.json	Stores fact → action mapping.
config.py	Stores API keys and model configuration.

5. **Sample Input → Fact → Action**

User Command	Interpreted Fact	Triggered Action
I'm cold	`cold_weather`	turn_on_heater
It's hot enough	`hot_enough`	turn_off_heater
I'm leaving the house	`leaving_home`	lock_door
The room is dark	`dark_room`	turn_on_lights
It's morning	`wake_up_time`	open_curtains

6. Conclusion

- The system demonstrates the use of **propositional logic** to drive intelligent behavior in smart homes.
- It successfully interprets natural language using a large language model and maps them to executable actions.
- The design is **modular** and **extensible** — new devices and rules can be added with minimal code changes.